MODELACION Y DISENO DE SISTEMAS 2026

Inicio ►

Colapsar todo

Pensando escuela.
Construimos esperanza
Analista Universitario en
Sistemas Informáticos
ESCMB
UNC

MODELACIÓN Y DISEÑO DE SISTEMAS

Ing. Fernando BONO

Avisos

Grupo de Whatsapp

Como contacto rápido relacionado a la materia, existe un grupo de Whatsapp en el cual se encuentra el docente y los alumnos interesados, en el mismo se puede consultar sobre las actividades, dudas sobre la materia, consulta sobre las clases dictadas.

Para acceder e ingresar al grupo de la Materia entrar a este link

Programa

Cronograma y Planificación

Hora	Lunes	Martes	Miércoles	Jueves	Viernes
17:30-18:10					
18:10-18:50					
19:00-19:40				CLASES Aula 32	
19:40-20:20				CLASES Aula 32	
20:20-21:00					CLASES Aula I33
21:00-21:40					CLASES Aula I33

💻 Clases Remotas con cuentas @miunc

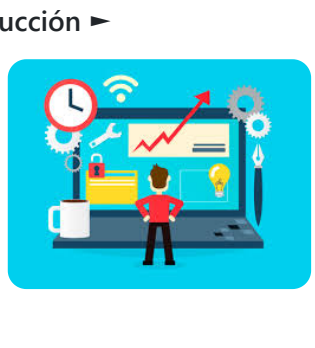
Las Clases remotas se realizarán por el siguiente enlace:

Enlace a la videollamada: https://meet.google.com/tcy-ezmh-kpn

IMPORTANTE: SI NO HAY AL MENOS EL 80% de los alumnos con la cámara prendida y participando de la clase, la misma se suspende automáticamente.

Bibliografía

Unidad 1 - Introducción ►



Importancia de modelar:

En el desarrollo de software es fundamental trabajar con modelos, un modelo es una representación de una perspectiva del software que nos permite visualizar el sistema, tratando de captar lo relevante definiendo puntos de vista que ayudan a simplificar la realidad y de esta manera reducir la complejidad inherente a los sistemas.

- ¿Por qué Modelamos?

Para conocer el dominio, descubrir requisitos, diseñar, dejar asentadas decisiones tomadas, mejorar la usabilidad, explorar soluciones, organizar el desarrollo, definir estructura, definir comportamiento...

Introducción

La Importancia de Modelar

Unidad 2 - Metodologías ►

En esta unidad se realiza una repaso de las metodologías tradicionales y las metodologías agiles para el desarrollo de software. Proceso Unificado de desarrollo de Software: Fases de desarrollo, Flujos de trabajo con sus actividades, trabajadores y artefactos. Metodologías Agiles: Concepto de agilidad. El manifiesto ágil. Los principios del manifiesto ágil. Comparativa entre metodologías ágiles y tradicionales. Historias de usuario. Casos en los que es conveniente aplicar metodologías ágiles. Scrum. El equipo. Sprints. Etapas. Valoración. Selección. Desarrollo. Revisión. Programación extrema (XP), Bases. Prácticas. Desarrollo dirigido por pruebas (TDD). Desarrollo dirigido por un plan y desarrollo. Administración de un proyecto ágil.

Proceso Unificado de Desarrollo

Ciclo de Vida del Software

Metodologias Basadas en Plan - Cascada - Espiral - Iterativo Incremental

Metodologias Ágiles - SCRUM

Unidad 3 - Ingenieria de Requerimientos ►

En esta unidad se aborda el proceso de ingeniería de requerimientos, que es fundamental en el ciclo de vida del desarrollo de software. Se exploran los métodos para obtener, validar y definir los requerimientos que el sistema debe cumplir. Se diferencia entre requerimientos funcionales y no funcionales, y se profundiza en cómo estos guían el diseño, la implementación y la validación del sistema.

Proceso de Ingeniería de Requerimientos: El proceso comienza con la obtención de requerimientos a través de la observación, reuniones con los interesados y la generación de prototipos. Se valida la información obtenida y se organiza en un documento consolidado que refleja las necesidades del cliente. A continuación, los requerimientos se priorizan y especifican claramente, asegurando que sean necesarios, verificables, completos, correctos y viables.

Tipos de Requerimientos:

- **Requerimientos funcionales**: Describen cómo debe comportarse el sistema ante un estímulo y la interacción entre el sistema y su entorno.
- **Requerimientos no funcionales**: Son restricciones sobre el sistema, como rendimiento, seguridad y fiabilidad.
- **Requerimientos de dominio**: Reflejan las características del dominio de aplicación del sistema.

Problemas Comunes con los Requerimientos: La ingeniería de requerimientos enfrenta desafíos como ambigüedad, contradicciones, falta de priorización y requisitos incompletos. Es crucial aplicar un proceso disciplinado para solucionar estos problemas y asegurar que el sistema se desarrolle según lo acordado.

Productos del Proceso de Ingeniería de Requerimientos: Los productos clave incluyen la lista consolidada de requerimientos, que debe ser clara y estar escrita en el lenguaje del cliente. Además, se utilizan prototipos desechables y escenarios para validar atributos de calidad del sistema.

Este enfoque asegura que los sistemas sean diseñados y construidos de acuerdo con las expectativas y necesidades de los usuarios, con una comunicación continua durante todo el proceso.

Ingeniería de Requerimientos

Requerimientos de Software Paso a Paso

Técnicas de Elicitación de Requerimientos

Administración de Requerimientos

Administración de Requerimientos - Complemento

Validación de Requerimientos

Protipado

Historias de usuario

Como evoluciona lo de OAS

Refinado del Backlog

Sprint - DoR - DoD

Documentación complementaria opcional de Lectura

1-sugerencias para leer paper

Paper 1- Elicitación de requisitos no funcionales basada en la gestión de conocimiento de los stakeholders

Paper 2- Proceso y evolución de los métodos formales en la ingeniería de requisitos

Paper 3- Proceso de especificacion de usabilidad con lel y escenarios partiendo del modelo de negocio

Paper 4- Explicitar Requisitos del Software usando Escenarios

Paper 5- Calidad de Software y Metodologías Ágiles en las PYMES de la Industria del Software

Actividad sobre Obtencion de Requerimientos de Software
Cierre: jueves, 30 de abril de 2026, 23:59

Unidad 4 - UML ►

UNIFIED
MODELING
LANGUAGE™
UML®

Modelos estáticos y dinámicos. Modelos de contexto. Modelos de interacción. Modelos estructurales. Modelos de comportamiento. Vistas. Ingeniería de Software Orientado a Objeto. Análisis orientado a objetos. Diagramas de Clases. Generalización y agregación.

Lenguaje Unificado de Modelado - UML. Casos de uso. Diagramas de actividad. Diagramas de clases. Diagramas de secuencia. Diagramas de estados. Modelado de sistemas con UML.

🔍 📊 📋 ETAPA DE ANÁLISIS

Comprensión del problema, relevamiento y modelado del dominio

Modelación de Sistemas orientados a Objetos

UML - Diagrama de Casos de Uso

UML - Diagrama de Objetos del Dominio

UML - Diagrama de Secuencias de Sistema

UML - Diagrama de Estados

🧩 📝 🔎 EJERCITACIÓN DE ANÁLISIS

Actividades prácticas de relevamiento y modelado

Ejercicio-01-ANALISIS-Estacionamiento
Cierre: sábado, 1 de agosto de 2026, 00:00

Ejercicio-02-ANALISIS-Pizzeria-Delivery
Cierre: sábado, 1 de agosto de 2026, 00:00

Ejercicio-03-ANALISIS-Biblioteca
Cierre: sábado, 1 de agosto de 2026, 00:00

Ejercicio-04-ANALISIS-Turnos-Medicos
Cierre: sábado, 1 de agosto de 2026, 00:00

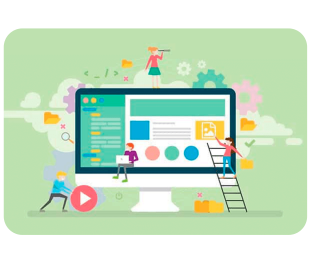
Ejercicio-05-ANALISIS-Cine
Cierre: sábado, 1 de agosto de 2026, 00:00

📚 📄 🧩 DOCUMENTACIÓN COMPLEMENTARIA

Material de referencia y estándares oficiales utilizados en la materia

Normativa Oficial UML

Unidad 5 - Usabilidad ►



Usabilidad. Antecedentes. Características principales de la usabilidad. Definiciones y conceptos. La Usabilidad en el Proceso Desarrollo de Software. La Interfaz y su importancia. Interacción Humano Computadora. El Factor Humano. Tipos de usuarios. Métodos de evaluación de la usabilidad. Usabilidad y Accesibilidad. Diseño Centrado en el Usuario. Análisis de Requerimientos en el Diseño Centrado en el Usuario. La Ingeniería de la Usabilidad. El proceso de ingeniería de usabilidad. Conceptos de Diseño. Heurística de Diseño.

Usabilidad

Actividad sobre Usabilidad
Cierre: jueves, 13 de agosto de 2026, 18:50

Actividad sobre Usabilidad Mercado Libre
Cierre: miércoles, 30 de septiembre de 2026, 23:59

Unidad 6 - Diseño ►

Introducción a la Etapa de Diseño en Ingeniería de Software: En esta etapa, se traduce la información obtenida durante el análisis de requerimientos en una estructura que guiará la implementación del sistema. El diseño de software es un proceso iterativo que comienza con una visión general del sistema y se va refinando hacia detalles más específicos, hasta alcanzar una representación detallada que permita su implementación.

Principales Objetivos del Diseño: El objetivo principal del diseño es crear una solución que sea resistente, funcional y estéticamente agradable para el usuario. Esto implica aplicar principios como la modularidad, la reutilización de componentes y la separación de preocupaciones. El diseño debe ser un plano bien estructurado que aborde diversos aspectos del sistema, como los datos, la arquitectura, las interfaces y los componentes.

Proceso de Diseño: El proceso de diseño involucra diversas actividades que incluyen la creación de la arquitectura del sistema, la descomposición en subsistemas, la definición de mecanismos de comunicación y la creación de los componentes. Además, se debe poner especial atención a la usabilidad de las interfaces, tanto internas como externas, para asegurar que el sistema sea fácil de utilizar y mantener.

Principios de Diseño: Entre los principios clave que guían el diseño de software se encuentran la alta cohesión, el bajo acoplamiento y la modularidad. Se deben aplicar patrones de diseño reconocidos que permitan la creación de sistemas reutilizables y fáciles de mantener. El diseño también debe estar orientado a la calidad, con un enfoque en la reducción de riesgos y la adaptación a cambios.

Consideraciones Importantes: El diseño debe ser evaluado continuamente durante su desarrollo para asegurar que cumpla con los requisitos funcionales y no funcionales del sistema. Además, se debe ser flexible y estar preparado para adaptarse a los cambios que puedan surgir durante el ciclo de vida del software.

Documentación y Comunicación: Es fundamental documentar adecuadamente el diseño para garantizar que todas las partes involucradas en el desarrollo (desarrolladores, testers, stakeholders) puedan entender la solución propuesta. La comunicación clara y eficaz entre los miembros del equipo de desarrollo es esencial para el éxito del proyecto.

Este enfoque permite asegurar que el software final no solo cumpla con los requisitos del usuario, sino que también sea fácil de mantener, expandir y utilizar.

🏗️ 💻 ⚙️ ETAPA DE DISEÑO

Diseño de la solución y comportamiento interno del sistema

UML - Diagrama de Clases

UML - Diagrama de Secuencias

D01 - Análisis → Diseño + Principios

D02 - Patrones GRASP (5 básicos + 4 avanzados)

D03 - Patrones GoF (Singleton, Observer, Strategy, Factory, Decorator, Adapter)

Informacion Adicional Complementaria

⚙️ 🏗️ 💻 EJERCITACIÓN DE DISEÑO

Actividades prácticas de diseño y construcción de modelos

Documento Patrones - blog Daniel Garcia

Investigacion sobre Patrones de Diseño
Cierre: jueves, 13 de agosto de 2026, 18:40

Ejercicio-06-DISEÑO-Estacionamiento
Cierre: sábado, 1 de agosto de 2026, 00:00

Ejercicio-07-DISEÑO-Pizzeria-Delivery
Cierre: sábado, 1 de agosto de 2026, 00:00

Ejercicio-08-DISEÑO-Biblioteca
Cierre: sábado, 1 de agosto de 2026, 00:00

Ejercicio-09-DISEÑO-Turnos-Medicos
Cierre: sábado, 1 de agosto de 2026, 00:00

Ejercicio-10-DISEÑO-Cine
Cierre: sábado, 1 de agosto de 2026, 00:00

Unidad 7 - Arquitectura ►

La **arquitectura de software** es una de las fases más críticas en el desarrollo de sistemas complejos, ya que establece la estructura fundamental sobre la cual se construirá el sistema. En esta etapa, se toman decisiones clave que afectan no solo la implementación, sino también la escalabilidad, la seguridad y el rendimiento del software a lo largo de su ciclo de vida.

Objetivos Principales de la Arquitectura de Software: La arquitectura tiene como objetivo organizar el sistema en **componentes** y definir las relaciones entre ellos. Además, debe proporcionar una estructura que permita la fácil integración de los componentes y el control de las interacciones entre ellos. Las decisiones tomadas durante esta fase buscan reducir los riesgos asociados con la construcción del sistema, permitiendo identificar posibles problemas antes de que se realicen inversiones en el desarrollo detallado.

Definición y Componentes de la Arquitectura: La arquitectura de software se considera la **organización fundamental del sistema**, representando cómo sus componentes interactúan entre sí y con el entorno. Esta definición implica que los sistemas deben ser estructurados de manera coherente y que cada componente debe tener responsabilidades claras. Los componentes son las unidades de procesamiento, mientras que los **conectores** permiten la comunicación y coordinación entre ellos.

Estilos Arquitectónicos: Los estilos arquitectónicos proporcionan plantillas de construcción reutilizables, cada una adaptada a diferentes tipos de sistemas y necesidades. Ejemplos de estilos arquitectónicos incluyen **cliente-servidor**, **arquitectura en capas**, **Model-View-Controller (MVC)**, **arquitecturas basadas en eventos**, **tubería-filtros**, y **arquitectura orientada a servicios**. Cada uno de estos estilos define cómo se estructuran los componentes y cómo interactúan entre sí, ayudando a los arquitectos a elegir el enfoque adecuado según los requerimientos del proyecto.

Vistas de la Arquitectura: Para representar de manera efectiva la arquitectura, se utilizan **vistas arquitectónicas**. Estas vistas incluyen diferentes perspectivas del sistema, como la **vista lógica**, que describe los módulos y sus interacciones, la **vista de procesos**, que aborda los aspectos de concurrencia y comunicación, y la **vista de despliegue**, que describe la infraestructura física sobre la que el sistema se ejecutará Cada vista proporciona información específica para los diferentes stakeholders, como desarrolladores, testers y gerentes de proyectos.

Importancia de la Documentación Arquitectónica: La documentación arquitectónica juega un papel crucial en la comunicación entre los equipos de trabajo y con los stakeholders del proyecto. Los documentos deben ser claros, concisos y estar actualizados para reflejar las decisiones tomadas a lo largo del proceso de desarrollo. Además, la documentación debe facilitar la comprensión del sistema para su posterior mantenimiento y evolución.

En resumen, la **arquitectura de software** no solo define la estructura de un sistema, sino que también influye directamente en su calidad, rendimiento y capacidad de adaptación futura. Un diseño arquitectónico sólido es la base sobre la cual se construye el sistema, y su correcta implementación permite la creación de soluciones flexibles, escalables y de alta calidad.

🏛️ 🌐 🏗️ ETAPA DE ARQUITECTURA

Estructura macro del sistema, estilos arquitectónicos y atributos de calidad

D04 - Arquitectura: conceptos + atributos de calidad + ADRs

D05 - Estilos clásicos: Capas + Cliente-Servidor

D06 - Estilo MVC

D07 - Microservicios

D08 - Event-Driven

D09 - Arquitecturas modernas: Clean + Hexagonal + Onion

Informacion Adicional Complementaria

🏛️ 📐 🌐 EJERCITACIÓN DE ARQUITECTURA

Actividades prácticas de arquitectura, estilos y decisiones (ADRs)

Ejercicio-11-ARQUITECTURA-Estacionamiento
Cierre: jueves, 1 de octubre de 2026, 00:00

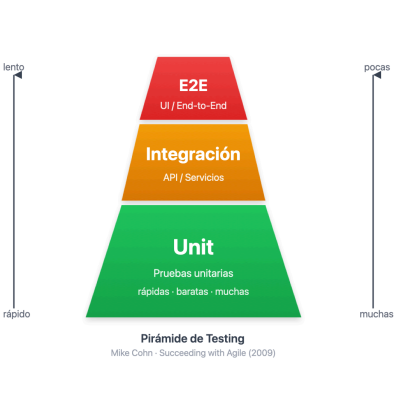
Ejercicio-12-ARQUITECTURA-Pizzeria-Delivery
Cierre: jueves, 1 de octubre de 2026, 00:00

Ejercicio-13-ARQUITECTURA-Biblioteca
Cierre: jueves, 1 de octubre de 2026, 00:00

Ejercicio-14-ARQUITECTURA-Turnos-Medicos
Cierre: jueves, 1 de octubre de 2026, 00:00

Ejercicio-15-ARQUITECTURA-Cine
Cierre: jueves, 1 de octubre de 2026, 00:00

Unidad 8 - Testing



El **testing de software** es una de las disciplinas más determinantes en el desarrollo de sistemas modernos, ya que verifica que el producto cumpla con los requisitos funcionales y no funcionales, y previene defectos antes de que lleguen a producción. En esta etapa, se diseñan y ejecutan estrategias que aseguran no solo la corrección del software, sino también su calidad, mantenibilidad y robustez a lo largo del tiempo.

Objetivos Principales del Testing de Software: El testing tiene como objetivo **detectar defectos** y validar el comportamiento del sistema en distintos escenarios. Además, debe proporcionar un marco que permita verificar la integración de los componentes y la satisfacción del usuario final. Las decisiones tomadas durante esta fase buscan reducir el riesgo de fallos en producción, permitiendo identificar problemas en etapas tempranas, donde corregirlos es significativamente más barato.

Definición y Tipos de Pruebas: El testing de software se considera el **proceso sistemático de verificación y validación del sistema**, representando cómo se comprueba que cada parte funciona como se espera y que el conjunto satisface las necesidades del negocio. Esta definición implica que las pruebas deben estar estructuradas y que cada una debe tener un propósito claro. Los **casos de prueba** son las unidades de verificación, mientras que las **suites** permiten organizar y ejecutar las pruebas de manera coordinada.

Niveles y Tipos de Testing: Los niveles de testing proporcionan estrategias complementarias, cada una adaptada a diferentes alcances del sistema. Ejemplos de niveles incluyen **pruebas unitarias**, **pruebas de integración**, **pruebas de sistema (E2E)**, **pruebas de aceptación (UAT)**, **pruebas de regresión**, y **pruebas no funcionales (performance, seguridad, usabilidad).** Cada uno de estos niveles define qué se prueba, con qué granularidad y en qué momento del ciclo de vida, ayudando a los equipos a elegir el enfoque adecuado según los requerimientos del proyecto.

Estrategias y Pirámide de Testing: Para organizar de manera efectiva el esfuerzo de pruebas, se utiliza la **pirámide de testing**. Esta estrategia propone diferentes proporciones de pruebas según su costo y velocidad: una **base amplia de pruebas unitarias**, que son rápidas y baratas, una **capa intermedia de pruebas de integración**, que verifica la colaboración entre componentes, y una **cima reducida de pruebas end-to-end**, que validan flujos completos de usuario. Cada capa proporciona confianza distinta y complementaria, permitiendo balancear cobertura, costo y tiempo de feedback.

Importancia de la Automatización y CI/CD: La automatización de pruebas juega un papel crucial en los equipos modernos. Las pruebas automatizadas deben ser confiables, rápidas y mantenibles, y formar parte de un pipeline de **integración y entrega continua (CI/CD)** que ejecute la suite en cada cambio. Además, deben ofrecer feedback inmediato para que los defectos se corrijan cuando todavía son baratos de reparar y el contexto está fresco en la cabeza del desarrollador.

En resumen, el **testing de software** no solo detecta errores, sino que también influye directamente en la calidad del producto, la confianza del equipo y la velocidad de entrega. Una estrategia de pruebas sólida es la red de seguridad sobre la cual se construye un sistema profesional, y su correcta implementación permite la evolución continua del software con bajo riesgo de regresión y alta confianza para los usuarios.

🧪 🔍 ✅ ETAPA DE TESTING

Verificación, validación y aseguramiento de la calidad del software

T01 - Fundamentos de Testing + Pirámide + estrategias

T02 - Pruebas Unitarias + TDD + Mocks

T03 - Pruebas de Integración + Contract Testing

T04 - Pruebas End-to-End + UAT

T05 - Pruebas no funcionales: Performance, Seguridad, Usabilidad

T06 - CI/CD + Pipelines + Entrega continua

PROYECTO GUÍA EN C#

🧪 💻 PROYECTO GUÍA EN C#

Las seis clases de testing, implementadas y funcionando sobre nuestro caso

🧪 🛠️ 📊 EJERCITACIÓN DE TESTING

Actividades prácticas de testing, automatización y aseguramiento de calidad

Ejercicio-16-TESTING-Estacionamiento
Cierre: jueves, 1 de octubre de 2026, 00:00

Ejercicio-17-TESTING-Pizzeria-Delivery
Cierre: jueves, 1 de octubre de 2026, 00:00

Ejercicio-18-TESTING-Biblioteca
Cierre: jueves, 1 de octubre de 2026, 00:00

Ejercicio-19-TESTING-Turnos-Medicos
Cierre: jueves, 1 de octubre de 2026, 00:00

Ejercicio-20-TESTING-Cine
Cierre: jueves, 1 de octubre de 2026, 00:00

Unidad 9 - BPMN ►

Introducción a BPMN:

BPMN (Business Process Model and Notation) es una notación gráfica estandarizada que se utiliza para modelar y representar los procesos de negocio de una organización. Su objetivo principal es proporcionar una manera clara y comprensible de visualizar la secuencia de actividades y los mensajes que fluyen entre los participantes en un proceso de negocio.

Definición de Procesos de Negocio: Un proceso de negocio se define como un conjunto de actividades relacionadas, estructuradas en una secuencia específica, que producen un servicio o producto con el fin de cumplir con un objetivo específico para un cliente. Estos procesos pueden ser automáticos o realizados por humanos y están orientados a mejorar la eficiencia de la organización.

Objetivos de BPM: El **Business Process Management (BPM)** tiene como objetivo mejorar la eficiencia y efectividad de los procesos empresariales mediante la gestión, diseño, modelado, documentación y optimización de los procesos a lo largo del tiempo. BPMN es una herramienta clave en este enfoque para representar y comunicar los procesos de negocio de forma comprensible.

Elementos Principales de BPMN:

1. **Objetos de Flujo:**
   - **Eventos**: Representan algo que sucede, y pueden ser iniciales (disparadores), intermedios (entre inicio y fin), o finales (terminan el proceso).
   - **Actividades**: Son tareas que se realizan dentro del proceso, ya sea de usuario, manual o de sistema.
   - **Compuertas de Control**: Deciden el flujo de trabajo, como bifurcaciones o combinaciones de rutas, dependiendo de las condiciones.
2. **Objetos de Conexión:**
   - **Flujo de Secuencia**: Muestra el orden en que se realizarán las actividades.
   - **Flujo de Mensaje**: Representa la comunicación entre participantes.
   - **Asociación**: Conecta artefactos o notas con elementos de flujo.
3. **Carriles y Piscinas (Swimlanes)**:
   - **Piscina (Pool)**: Representa a los participantes principales del proceso, generalmente diferentes organizaciones o entidades.
   - **Carril (Lane)**: Divide a los participantes dentro de una piscina según su rol o función, para categorizar las actividades.
4. **Artefactos**:
   - **Objetos de Datos**: Muestran los datos requeridos o producidos por una actividad.
   - **Grupos**: Agrupan actividades sin alterar el flujo de trabajo.
   - **Anotaciones**: Añaden explicaciones al modelo para hacerlo más comprensible.

Aplicaciones de BPMN:

- BPMN es utilizado en la documentación de procesos para mejorar la trazabilidad, la comunicación entre equipos de desarrollo y negocio, y la integración de procesos en sistemas más grandes.
- Es ampliamente utilizado en la automatización de procesos y la mejora de la eficiencia mediante herramientas como BPMS (Business Process Management Suites).

Términos Relacionados:

- **BPMS**: Software que ayuda en la gestión de procesos a lo largo de su ciclo de vida, desde el diseño hasta la ejecución y el análisis.
- **Workflow**: Secuencia de actividades que deben seguirse para completar una tarea.
- **RPA (Automatización Robótica de Procesos)**: Tecnología que utiliza robots de software para automatizar tareas repetitivas.

Modelación de Sistemas Orientados a Procesos - Fundamentos

BPMN - Bussiness Process Management Notacion

Proyecto ►

En el **trabajo integrador** transforman el caso de OAS de su grupo en un **proyecto listo para implementar**: backlog ágil, plan de sprints, casos de uso, arquitectura con sus **ADR**, diagramas **UML** con **patrones de diseño**, historias de usuario completas, plan de pruebas y un prototipo evaluado con heurísticas de usabilidad.
Todo vive en **Azure DevOps** o en **GitHub**, y un agente de IA revisa cada entrega y señala lo que falta. La nota la pone el docente: pueden usar IA, pero cada integrante tiene que poder defender el trabajo.

PROYECTO FINAL POR GRUPO (Parcial 2 y 3)
Cierre: lunes, 5 de octubre de 2026, 18:00

🧩 Trabajo integrador: alternativa en GitHub

Descripción: Microsoft ahora pide una suscripción de Azure para crear organizaciones nuevas de Azure DevOps, y eso trabó a varios grupos. Si su grupo todavía no tiene organización, puede hacer el trabajo en GitHub, que es gratis y no pide tarjeta. Los grupos que ya tienen su proyecto en Azure DevOps siguen ahí y no cambian nada.

Los pasos (el video y la guía los muestran uno por uno):

1. Crear la cuenta de GitHub (alcanza con una persona del grupo; cada integrante necesita su cuenta para ser invitado).
2. Crear un repositorio privado con README.
3. Invitar al docente al repositorio: Settings → Collaborators → fernandobono.
4. Crear el tablero (Projects → New project → Board).
5. Agregar los campos Sprint (Iteration), Puntos (Number) y Prioridad (Must, Should, Could, Won't).
6. Invitar al docente también al tablero: Settings → Manage access → fernandobono, rol Read.
7. Cargar el backlog: etiquetas épica, feature e historia; épica → feature → historia con «Create sub-issue».
8. Completar Sprint, Puntos y Prioridad de cada historia.

> ⚠️ **Importante:** inviten a fernandobono al repositorio y también al tablero. El tablero tiene su propio permiso: sin ese paso, el corrector no ve los sprints, los puntos ni las prioridades. Ojo con el usuario parecido fernandobonor-dot: no es el docente.

Qué suben a Moodle: el link del repositorio (https://github.com/usuario/repositorio), igual que el link de Azure DevOps en los otros grupos.

▶ Ver Video | 📖 Ver Guía paso a paso (PDF)

Extra - Como adaptar junto a la IA el proceso de desarrollo y las metodologías

IA Y PROCESO DE DESARROLLO
IA

El **Spec-Driven Development (SDD)** es un enfoque de desarrollo de software que propone definir de manera precisa las especificaciones antes de la implementación. En este modelo, la especificación se convierte en la fuente de verdad, guiando tanto la generación del código como su validación.

A diferencia de enfoques tradicionales, donde los errores se detectan en etapas posteriores, **SDD** busca anticiparlos mediante la definición de requisitos, criterios de aceptación y restricciones desde el inicio. Esto permite reducir el retrabajo, mejorar la calidad del software y lograr mayor trazabilidad entre lo que se necesita y lo que se construye.

Con la incorporación de herramientas basadas en **inteligencia artificial**, este enfoque se potencia, permitiendo automatizar parte del proceso de desarrollo y verificación, manteniendo coherencia entre la especificación y el sistema final.

Charla con CTO de Habi (primera proptech unicornio de Hispanoamérica), (Adrian Mastronardi)

SCRUM
MURIÓ
#155

Charla de Brian Madison el creador del método BMAD

BMad Method
MASTERCLASS
The Complete Workflow
CLAUDE
CODE

BMAD Method GitHub Repo: https://github.com/bmadcode/BMAD-METHOD

🤖 📐 BLOQUE 6 · DESARROLLO ASISTIDO POR IA

Cómo se le pide trabajo a una IA, y cómo se controla lo que entrega

Arriba está la idea general de SDD y dos charlas para ampliar. Acá van **las cuatro clases del bloque**. La IA ya programa, y rápido: el problema es que **inventa lo que no le dijimos** y **dice "listo" sin haberlo probado**. Vamos a ver las dos defensas: la **spec**, que fija qué significa "correcto" antes de escribir código, y el **arnés**, que decide si algo está terminado con evidencia y no con la palabra del modelo.

Todo lo que trabajamos en el año —requerimientos, historias con criterios, modelos, arquitectura, testing y CI/CD— **es justo lo que la IA necesita para no inventar**. Por eso este bloque va al final.

El bloque en cuatro clases

CLASE	FECHA	TEMA	AL TERMINAR VAS A PODER...
I01	Jue 8/10	Por qué la IA necesita una spec	Explicar por qué un pedido sin spec sale distinto cada vez, y escribir una spec mínima
I02	Vie 9/10	La spec y BMAD	Escribir una historia que la IA implemente sin inventar, y explicar qué hace BMAD con ella
I03	Jue 15/10	BMAD aplicado y el arnés durante el desarrollo	Explicar por qué la IA no puede controlarse sola y qué pieza del arnés ataja cada error
I04	Vie 16/10	El arnés después	Responder «¿cómo sé que está terminado?» sin decir «porque la IA dijo»

📌 **Parcial 3** · viernes 30/10 · El jueves 29/10 hacemos el repaso integrador.

Material de apoyo

🌐 Guía web: SDD, BMAD y arnés
bmad.netlify.app/

📁 Proyecto ChangoMás en C# (el del bloque de Testing, base de la demo)
github.com/fernandobono/changomas-testing-csharp

📘 Documentación oficial de BMAD
docs.bmad-method.org/

Modelación y Diseño de Sistemas · UNC — Escuela Superior de Comercio Manuel Belgrano — AUSI · Ing. Fernando Bono
Bloque 6 · Desarrollo asistido por IA · Caso hilo conductor: ChangoMás, cajas de autoservicio.

▯ Clase I01 · Por qué la IA necesita una spec

Descripción: Primera clase del bloque de desarrollo asistido por IA. Vemos los tres defectos de fábrica de un agente de IA (olvida, es inconsistente y dice «listo» sin probar) y cómo el Spec-Driven Development los ataca: definir el qué, el porqué y cómo se verifica, antes del código.

Cómo usarlo: El video narra la presentación completa. Tiene subtítulos en español y la voz es una clonación con IA de la voz del profesor; el contenido lo revisó y aprobó él.

📄 Presentación (PDF) | ▶ Ver Video | 💻 GitHub Spec Kit | 💻 Kiro

▯ Clase I02 · La spec y BMAD

Descripción: Cómo se escribe una spec que la IA no pueda malinterpretar (piezas, criterios, no-objetivos y la historia con su tabla de tests) y BMAD, la herramienta que la opera con un equipo de agentes. Incluye cómo usar BMAD gratis con Gemini, con la demo grabada.

Cómo usarlo: El video narra la presentación completa e incluye la demo grabada de Gemini armando una spec. Tiene subtítulos en español y la voz es una clonación con IA de la voz del profesor; el contenido lo revisó y aprobó él.

📄 Presentación (PDF) | ▶ Ver Video | ▶ Demo: Gemini arma la spec | 📓 Historia HU-014 (.md) | 📓 Spec que armó Gemini (.md) | 💻 BMAD en GitHub | 📘 Documentación de BMAD | 🔑 Clave gratis de AI Studio | 💻 Gemini CLI

▯ Clase I03 · BMAD aplicado y el arnés durante el desarrollo

Descripción: Implementamos la HU-014 (producto de peso variable) con Claude Code y BMAD sobre el proyecto C# de ChangoMás: plan, aprobación, tests primero y revisión, que encontró un cobro silenciosamente mal. Y vemos el arnés: lo que controla a la IA mientras trabaja (frenos, revisores independientes, ejecutar para refutar). Incluye un arreglo chico hecho gratis con Gemini.

Cómo usarlo: El video narra la presentación e incluye las dos demos grabadas de verdad. Tiene subtítulos en español y la voz es una clonación con IA de la voz del profesor; el contenido lo revisó y aprobó él. Los archivos .md y .diff son la historia, el plan, los pendientes y los cambios reales que hizo la IA: abrilos con cualquier editor de texto.

📄 Presentación (PDF) | ▶ Ver Video | ▶ Demo: Claude Code + BMAD (HU-014) | ▶ Demo: arreglo con Gemini gratis | 📓 Historia HU-014 (.md) | 📓 Plan que armó BMAD (.md) | 📓 Pendientes declarados (.md) | 📓 Cambios de código HU-014 (.diff) | 📓 Cambios del arreglo con Gemini (.diff) | 💻 Proyecto ChangoMás en C# | 💻 BMAD en GitHub | 📘 Claude Code | 💻 Gemini CLI

▯ Clase I04 · El arnés después: controlar lo que la IA entregó

Descripción: Última clase del bloque. Cómo se controla lo que la IA entregó: afirmación contra evidencia, la matriz por criterio, la Definition of Done verificable, tests de mutación, CI como portón, despliegue con aprobación humana y los controles que corren de noche. Cerramos con el mapa completo del arnés y lo que pasó en nuestra propia demo de ChangoMás.

Cómo usarlo: El video narra la presentación completa. Tiene subtítulos en español y la voz es una clonación con IA de la voz del profesor; el contenido lo revisó y aprobó él. La actividad (el mapa del arnés de ChangoMás) entra en el Parcial 3.

📄 Presentación (PDF) | ▶ Ver Video | 📓 Ejemplo real: pendientes declarados (.md) | 💻 El pipeline de CI de ChangoMás | 📘 Tests de mutación (Stryker.NET)

🌐 Guía web: SDD, BMAD y arnés

Descripción: Todo el bloque de desarrollo asistido por IA en una sola página para leer y repasar: las cuatro clases con sus videos, las demos grabadas de verdad, cómo usar BMAD gratis con Gemini, la instalación paso a paso, un glosario y todas las descargas.

Cómo usarla: Abrila desde la computadora o el celular. Cada clase tiene el video, la presentación en PDF y las ideas centrales. Sirve para repasar antes del Parcial 3.

🌐 Abrir la guía web

🛒 Caso conductor: ChangoMás, cajas de autoservicio

Descripción: Es el caso que recorre toda la cursada: las cajas de autoservicio de una cadena de supermercados, donde el cliente escanea, paga y se va con mínima asistencia. Nació de un trabajo de campo real de OAS y lo usamos para todo: requerimientos, historias de usuario, UML, arquitectura, testing y desarrollo asistido por IA. Acá tenés el trabajo original, la ficha con el caso ordenado y el backlog con las 3 épicas y las 27 historias de usuario.

Cómo usarlo: Empezá por la ficha estructurada: resume el negocio, los procesos, los requerimientos y las reglas. El backlog trae las historias con su prioridad y su sprint (por ejemplo, HU-013 quitar un producto y HU-014 producto de peso variable, las del bloque de IA). El proyecto en C# implementa las historias del Sprint 1.

📄 Ficha estructurada del caso | 📋 Backlog inicial (épicas e historias) | 📄 Trabajo de campo OAS original | 💻 Proyecto ChangoMás en C#